Quiz 1

BRAC UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ I CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [20 points]

- By default, the condition of a laptop is “good”.
- Assume a tech shop can add a maximum of 10 laptops.
- Consider the condition of a laptop to be private.

Tester Code	Output

public class TechTester {
  public static void main(String[] args) {
  Laptop l1= new Laptop("Intel","I7","8 GB");
  Laptop l2= new Laptop("Intel","I9","16 GB");
  Laptop l3= new Laptop("AMD","Ryzen 6","16 GB");
  System.*out*.println("1==========");
  System.*out*.println(l1.details());
  System.*out*.println("2==========");
  TechShop shop1 = new TechShop("TechFrantic");
  System.*out*.println("3==========");
  shop1.addLaptop(l1, l2);
  shop1.addLaptop(l3);
  System.*out*.println("4==========");
  shop1.shopDetails();
  System.*out*.println("5==========");
  System.*out*.println(shop1.sellLaptop(l2));
  System.*out*.println("6==========");
  shop1.shopDetails();
  System.*out*.println("7==========");
  l3.markDeffective();
  System.*out*.println(l3.details());
  System.*out*.println("8==========");
  System.*out*.println(shop1.sellLaptop(l3));
  System.*out*.println("9==========");
  shop1.shopDetails();
  System.*out*.println("10==========");
  System.*out*.println("Recent sold: "+shop1.recentSold.details());
  }
}
	1==========
	Intel-I7-8 GB-Condition: good
	2==========
	A tech shop is created named TechFrantic!
	3==========
	Added in stock.
	Added in stock.
	Added in stock.
	4==========
	Shop name: TechFrantic
	No of laptops in stock: 3
	Intel-I7-8 GB-Condition: good
	Intel-I9-16 GB-Condition: good
	AMD-Ryzen 6-16 GB-Condition: good
	5==========
	A laptop has been sold!
	6==========
	Shop name: TechFrantic
	No of laptops in stock: 2
	Intel-I7-8 GB-Condition: good
	AMD-Ryzen 6-16 GB-Condition: good
	7==========
	AMD-Ryzen 6-16 GB-Condition: defective
	8==========
	Item defective. Cannot sell.
	9==========
	Shop name: TechFrantic
	No of laptops in stock: 2
	Intel-I7-8 GB-Condition: good
	AMD-Ryzen 6-16 GB-Condition: defective
	10==========
	Recent sold: Intel-I9-16 GB-Condition: good

Solution:

```
public class Laptop {
  public String company;
  public String processor;
  public String RAM;
  private String condition;
  public Laptop(String a, String b, String c){
     company = a;
     processor = b;
     RAM = c;
     condition = "good";
  }
  public String getCondition(){
     return condition;
  }
  public String details(){
     String info = String.format("%s-%s-%s-Condition: %s", company, processor, RAM, condition);
     return info;
  }
  public void markDeffective(){
     condition = "defective";
  }
}

public class TechShop {
  public Laptop recentSold;
  public Laptop stockItems [] = new Laptop[10];
  int index, count;
  public String name;
  public TechShop(String n){
     name = n;
     System.out.printf("A tech shop is created named %s!\n", name);
  }
  public void addLaptop(Laptop obj){
     if (index< stockItems.length){
        stockItems[index++] = obj;
        System.out.println("Added in stock.");
        count++;
     }
  }
  public void shopDetails(){
     System.out.println("Shop name: "+name);
     System.out.println("No of laptops in stock: "+count);
     for (int i =0; i<index; i++){
        if (stockItems[i] != null) System.out.println(stockItems[i].details());
     }
  }
  public String sellLaptop(Laptop obj){
```

```
        if(obj.getCondition().equals("good")){
            recentSold = obj;
            count--;
            for (int i=0;i<index;i++){
                if (stockItems[i] == obj){
                    stockItems[i] = null;
                }
            }
            return "A laptop has been sold!";
        }
        return "Item defective. Cannot sell.";
    }
    public void addLaptop(Laptop obj1, Laptop obj2){
        addLaptop(obj1);
        addLaptop(obj2);
    }
}
```

Quiz 1 Sec15

BRAC UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ I CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [20 points]

- A road is considered safe if it's **accidentCount <= 5 AND lighting >= 4**.
- checkRoute() method checks if a road is unsafe for driving. If not, redirect to an alternative safe route closest to the unsafe road.
  Ex: DarkStreet is not safe, and so it is redirected to the safest road closest to it - HighwayA. If more than one safe road is found with the closest distance, redirect to any one road.
- In case of emergency, you can **continue driving normally if the road is safe**. If the road is **unsafe but accidentCount <= 7 and lighting >=3**, an emergency route may be allowed. Else, **redirect to an alternate safe route**. Ex: HillWay's is an unsafe road but an emergency route was allowed as its accidentCount is 6 which is <=7 and lighting is 5 which is >= 3.

Tester Code

```
public class NavigationTester {
   public static void main(String[] args) {
      Road r1 = new Road("HighwayA",2,7);
      Road r2 = new Road("DarkStreet",8,2);
      Road r3 = new Road("RiverRoad",3,6);
      Road r4 = new Road("HillWay",6,5);
      Road r5 = new Road("MarketRoad",8,5);
      r1.roadDetails();
      System.out.println("1==============");
      NavigationSystem n1 = new NavigationSystem("CitySafe");
      System.out.println("2==============");
      n1.addRoad(r1);
      n1.addRoad(r2);
      n1.addRoad(r3);
      n1.addRoad(r4);
      n1.addRoad(r5);
      System.out.println("3==============");
      n1.systemReport();
      System.out.println("4==============");
      n1.checkRoute(r1,"Emergency");
      System.out.println("5==============");
      n1.checkRoute(r4,"Emergency");
      System.out.println("6==============");
      n1.checkRoute(r5,"Emergency");
      System.out.println("7==============");
      n1.checkRoute(r3);
      System.out.println("8==============");
      n1.checkRoute(r2);
   }  }
```

Output

```
Road Name: HighwayA
Accident Count: 2
Lighting Level: 7
1==============
Navigation System CitySafe activated.
2==============
3==============
Navigation System: CitySafe
Total Roads: 5/10
HighwayA : Safe
DarkStreet : Unsafe
RiverRoad : Safe
HillWay : Unsafe
MarketRoad : Unsafe
4==============
Route HighwayA is safe. Continue driving.
5==============
Emergency route allowed via HillWay
6==============
Warning! MarketRoad is unsafe.
Searching for alternate route...
Redirecting to RiverRoad
7==============
Route RiverRoad is safe. Continue driving.
8==============
Warning! DarkStreet is unsafe.
Searching for alternate route...
Redirecting to HighwayA
```

Solution:

```
public class Road {
  public String name;
  public int accidents;
  public int lighting;
  public Road(String n, int a, int b){
     name = n;
     accidents = a;
     lighting = b;
  }
  public void roadDetails()
  {
     System.out.println("Road Name: "+name);
     System.out.println("Accident Count: "+accidents);
     System.out.println("Lighting Level: "+lighting);
  }
}

public class NavigationSystem {
  public String systemName;
  public Road[] roadObjects;
  public boolean[] safeRoads;
  public int count;
  public NavigationSystem(String systemName) {
     this.systemName = systemName;
     roadObjects = new Road[10];
     safeRoads = new boolean[10];
     System.out.println("Navigation System " + systemName + " activated.");
  }
  public void addRoad(Road obj) {
     roadObjects[count] = obj;
     if (obj.accidents < 5 && obj.lighting >= 4) {
        safeRoads[count] = true;
     } else {
        safeRoads[count] = false;
     }
     count++;
  }
  public void checkRoute(Road obj) {
     checkRoute(obj, "Not emergency");
  }
  public void checkRoute(Road obj, String urgencyLevel) {
     int index = -1;
     for (int i = 0; i < count; i++) {
        if (roadObjects[i].name.equals(obj.name)) {
           index = i;
           break;
        }
```

```java
        }
        if (safeRoads[index]) {
            System.out.println("Route " + obj.name + " is safe. Continue driving.");
            return;
        }
        if (urgencyLevel.equals("Emergency") && obj.accidents <= 7 && obj.lighting >= 3) {
            System.out.println("Emergency route allowed via " + obj.name);
            return;
        }
        System.out.println("Warning! " + obj.name + " is unsafe.");
        System.out.println("Searching for alternate route...");
        for (int i = index - 1, j = index +1; i >= 0 || j<count; i--, j++) {  // searching for road nearest to current road
            if (safeRoads[i]) {
                System.out.println("Redirecting to " + roadObjects[i].name);
                return;
            }
            if (safeRoads[j]) {
                System.out.println("Redirecting to " + roadObjects[i].name);
                return;
            }
        }
    }
    public void systemReport() {
        System.out.println("Navigation System: " + systemName);
        System.out.println("Total Roads: " + count +"/" + roadObjects.length);
        for (int i = 0; i < count; i++) {
            System.out.print(roadObjects[i].name + " : ");
            if (safeRoads[i]) System.out.println("Safe");
            else System.out.println("Unsafe");
        }
    }
}
```

Quiz 2

BRAC UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ II CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [10 points]

Trace the output of the following code and write the outputs in the output boxes. 1 mark for each output (**9 outputs in total**) and 1 mark for trace table. You must show the proper working and **circle the obtained outputs in the trace table**.

```
public class Tracing {
  int a = 1, b = 9;
  int [] z;
  public Tracing(){
      a += b;
      b -= 1;
  }
  public void m1(int a){
     if (a % 2 != 0) {
        m2(this);
        return;
      }
      int [] z = {5, 6};
      z[0] = (a++) + this.b;
      this.z[1] = z[0] + this.z[0];
      System.out.println(a + " " + z[0] + " " + this.z[1]);
  }
  public Tracing(Tracing temp){
      this();
      b = temp.b + 5;
      this.z = new int[]{2, 4, 6};
      temp.z[2] = this.z[2]++;
      System.out.println(a + " " + b + " " + temp.z[2]);
  }
  public void m2(Tracing obj){
      int b = obj.z[0] + this.b;
      obj.a = a + 7;
      System.out.println(this.a + " " + b + " " + obj.z[2]);
  }
}
```

Illustrate the output of the following statements written in the main method of a tester class.

Driver code:

```
Tracing t1 = new Tracing();
t1.z = new int[]{6, -1, 7};
t1.m1(t1.b);
Tracing t2 = new Tracing(t1);
t2.m1(7);
```

Output:

BRAC UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ II. CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [10 points]

Trace the output of the following code and write the outputs in the output boxes. 1 mark for each output (**9 outputs in total**) and 1 mark for trace table. You must show the proper working and **circle the obtained outputs in the trace table**.

```
public class TracingB {
  int a = 5, b = 6;
  int [] z;
  public TracingB(){
      a += b;
      b -= 2;
  }
  public void m1(int a){
      if (a % 2 != 0) {
        m2(this);
        return;
      }
      int [] z = {8, 5};
      z[0] = (a++) + this.b;
      this.z[1] = z[0] + this.z[0];
      System.out.println(a + " " + z[0] + " " + this.z[1]);
  }
  public TracingB(TracingB temp){
      this();
      b = temp.b + 3;
      this.z = new int[]{2, 3, -1};
      temp.z[2] = this.z[2]++;
      System.out.println(a + " " + b + " " + temp.z[2]);
  }
  public void m2(TracingB obj){
      int b = obj.z[0] + this.b;
      obj.a = a + 2;
      System.out.println(this.a + " " + b + " " + obj.z[2]);
  }
}
```

Illustrate the output of the following statements written in the main method of a tester class.

Driver code:

```
TracingB t1 = new TracingB();
t1.z = new int[]{2, -2, 4};
t1.m1(t1.b);
TracingB t2 = new TracingB(t1);
t2.m1(5);
```

Output:

Solution:

Slot 1
9 16 22
10 13 6
17 15 7

Slot 2

5 8 10
11 7 -1
13 9 0

Quiz2 Sec15

BRAC UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ II CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [10 points]

Trace the output of the following code and write the outputs in the output boxes. 1 mark for each output (**9 outputs in total**) and 1 mark for trace table. You must show the proper working and **circle the obtained outputs in the trace table**.

```
public class TracingA {
  int x = 1, y = 8, z = 6;
  public TracingA(){
      x += y;
      y -= 1;
  }
  public TracingA(TracingA temp){
      y += temp.y;
      z = y + (temp.z--);
      System.out.println(y + " " + z + " " + temp.z);
  }
  public void m1(int x){
    if (x %2 != 0){
        TracingA temp = new TracingA();
        x = this.m2(temp) + temp.z;
        System.out.println(x + " " + this.y + " " + temp.x);
        return;
      }
      int [] z = {-1, 12};
      z[0] = (++x) + this.x;
      this.z = z[1] + x;
      System.out.println(x + " " + z[0] + " " + this.z);
  }
  public int m2(TracingA obj){
      obj.x = this.x + 5;
      return obj.x;
  }
}
```

Illustrate the output of the following statements written in the main method of a tester class.

Driver code:

```
TracingA t1 = new TracingA();
t1.m1(7);
new TracingA(t1).m1(8);
```

Output:

BRAC UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ II. CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [10 points]

Trace the output of the following code and write the outputs in the output boxes. 1 mark for each output (**9 outputs in total**) and 1 mark for trace table. You must show the proper working and **circle the obtained outputs in the trace table**.

```
public class TracingB {
  int x = 3, y = 5, z = 7;
  public TracingB(){
      x += y;
      y -= 1;
  }
  public TracingB(TracingB temp){
      y += temp.y;
      z = y + (temp.z--);
      System.out.println(y + " " + z + " " + temp.z);
  }
  public void m1(int x){
      if (x %2 != 0){
        TracingB temp = new TracingB();
        x = this.m2(temp) + temp.z;
        System.out.println(x + " " + this.y + " " + temp.x);
        return;
      }
      int [] z = {0, 15};
      z[0] = (++x) + this.x;
      this.z = z[1] + x;
      System.out.println(x + " " + z[0] + " " + this.z);
  }
  public int m2(TracingB obj){
      obj.x = this.x + 4;
      return obj.x;
  }
}
```

Illustrate the output of the following statements written in the main method of a tester class.

Driver code:

```
TracingB t1 = new TracingB();
t1.m1(5);
new TracingB(t1).m1(10);
```

Output:

Solution:

Slot 1
20 7 14
15 21 5
9 10 21

Slot 2

19 4 12
9 16 6
11 14 26

Quiz 3

BRAC UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

QUIZ III CSE111: PROGRAMMING LANGUAGE II

NAME:	**ID:**	**SECTION:**

Question - 1: [20 points]

Write the design class so that the following tester code produces the desired output.

- Assume there can be a maximum of 5 registered users.

Tester Code	Output

public class CloudStorageTester {
public static void main(String[] args) {
System.out.println("----------1----------");
CloudStorage.systemStats();
System.out.println("----------2----------");
CloudStorage u1 = new CloudStorage("hacker01");
u1.upload("doc", 120);
System.out.println("----------3----------");
CloudStorage.systemStats();
System.out.println("----------4----------");
u1.userDetails();
System.out.println("----------5----------");
CloudStorage u2 = new CloudStorage("coderX");
u2.upload("media", 1500);
System.out.println("----------6----------");
CloudStorage.systemStats();
System.out.println("----------7----------");
u2.upload("doc", 300);
System.out.println("----------8----------");
u2.userDetails();
System.out.println("----------9----------");
CloudStorage.systemStats();
}
}
	==========1==========
	Users: None
	Total Documents Storage: 0
	Total Media Files Storage: 0
	Total Storage Used: 0
	==========2==========
	hacker01 uploaded 120MB doc to cloud storage
	==========3==========
	Users: hacker01
	Total Documents Storage: 120 MB
	Total Media Files Storage: 0 MB
	Total Storage Used: 120 MB
	==========4==========
	hacker01's Cloud Storage:
	Documents: 120 MB
	Media Files: 0 MB
	==========5==========
	coderX uploaded 1500MB media to cloud storage
	==========6==========
	Users: hacker01, coderX
	Total Documents Storage: 120 MB
	Total Media Files Storage: 1500 MB
	Total Storage Used: 1620MB
	==========7==========
	coderX uploaded 300MB doc to cloud storage
	==========8==========
	coderX's Cloud Storage
	Documents: 300 MB
	Media Files: 1500 MB
	==========9==========
	Users: hacker01, coderX
	Total Documents Storage: 420 MB
	Total Media Files Storage: 1500 MB
	Total Storage Used: 1920MB

Solution:

```
public class CloudStorage
{
  public static String allUsers="";
  public static int count;
  public static int totalDoc;
  public static int totalMedia;
  public String name;
  public int doc;
  public int media;
  public CloudStorage(String name){
    this.name = name;
    if(count < 5){
      allUsers += name+" ";
      count++;
    }
  }
  public void upload(String type, int size) {
    System.out.println(name + " uploaded " + size + "MB " + type + " to cloud storage");

    if (type.equals("doc")) {
      doc += size;
      totalDoc += size;
    }
    else if (type.equals("media")) {
      media += size;
      totalMedia += size;
    }
  }
  public void userDetails() {
    System.out.println(name + "'s Cloud Storage:");
    System.out.println("Documents: " + doc + " MB");
    System.out.println("Media Files: " + media + " MB");
  }
  public static void systemStats() {
    System.out.print("Users: ");
    if (count == 0) {
      System.out.println("None");
    } else {
        System.out.println(allUsers);
    }
    System.out.println("Total Documents Storage: " + totalDoc + " MB");
    System.out.println("Total Media Files Storage: " + totalMedia + " MB");
    System.out.println("Total Storage Used: " + (totalDoc + totalMedia) + "MB");
```

```
  }
}
```